

The Unfish Programming Language: Architecture, Design, and Implementation

From Drag-and-Drop Visual Blocks to Bare-Metal Systems Programming

The Unfish Project Core Engineering Team

October 2026 - First Edition (v2.1.0)

Contents

1	Preface: The Principle of Intellectual Ascent	2
2	PART I: FOUNDATIONS & PEDAGOGICAL VISION	3
2.1	Chapter 1: The Crisis in Computing Education	3
2.1.1	1.1. The Toy Language Trap	3
2.1.2	1.2. The Industrial Cliff	3
2.1.3	1.3. The Black Box Deception	3
2.2	Chapter 2: The Glass Box Paradigm	3
3	PART II: LANGUAGE TOUR & CORE PROGRAMMING	5
3.1	Chapter 3: Lexical Structure, Indentation & Syntax	5
3.1.1	3.1. Clean Indentation Without Braces	5
3.1.2	3.2. Comments & Docstrings	5
3.2	Chapter 4: Variables, Types & Control Flow	5
3.2.1	4.1. Variables with <code>let</code>	5
3.2.2	4.2. Formatted String Interpolation (<code>f"..."</code>)	5
3.2.3	4.3. Loop Constructs	5
3.3	Chapter 5: Functions, Closures & <code>fn</code> Lambdas	5
3.3.1	Lexical Closures	6
3.4	Chapter 6: Structs, Methods & Ocean Simulation	6
3.5	Chapter 7: Structured Exception Handling	7
3.5.1	The Invariants of Exception Unwinding	7
4	PART III: SYSTEMS PROGRAMMING & COMPILER INTERNALS	8
4.1	Chapter 8: Contiguous Byte Buffers & Endian Codecs	8
4.2	Chapter 9: The Dual Virtual Machine Architecture	8
4.3	Chapter 10: Ahead-of-Time C99 Transpilation & Bare-Metal Hardware	8
4.3.1	Flashing Bare-Metal Microcontrollers	8
5	APPENDIX: FORMAL GRAMMAR & VM OPCODES	9
5.0.1	Core EBNF Grammar Excerpt	9
5.0.2	Complete Test Parity Verification Ledger	9

1 Preface: The Principle of Intellectual Ascent

Every programmer remembers the journey from their first “Hello, World!” to writing their first real data structure, debugging their first compiler error, or optimizing their first low-level algorithm.

For decades, however, that journey has been unnecessarily painful. Introductory programming education has been fractured into isolated, incompatible camps. Children begin with visual block tools like Scratch—drag-and-drop toys that remove syntax frustration but introduce artificial dead ends. When students transition to text, they hit a cognitive cliff: industrial languages like Python, Java, or C++ assault them with cryptic error messages, complex build systems, and punctuation noise. Worse, modern high-level languages treat the physical computer as a black box. Hardware is obscured behind gigabytes of runtime infrastructure. When students finally need to learn computer architecture, operating systems, and memory management, they are forced to switch to C, starting over from zero while spending 80% of their bandwidth fighting segmentation faults.

Unfish was designed to eliminate this fragmentation once and for all.

Unfish provides an **unbroken path of intellectual ascent**. In Unfish, a student can start by dragging visual blocks in elementary school, transition naturally into clean Python-like text in high school, study compiler construction and virtual machine design in college by reading the runtime’s clean ANSI C99 source code, and deploy high-performance algorithms to bare-metal ARM Cortex-M microcontrollers—using the **exact same language, the exact same mental models, and the exact same core runtime**.

This book is the definitive, comprehensive guide to the Unfish programming language, its multi-tier virtual machines, its optimizing compilers, and its systems programming capabilities.

2 PART I: FOUNDATIONS & PEDAGOGICAL VISION

2.1 Chapter 1: The Crisis in Computing Education

Modern software education suffers from three fundamental pathologies:

2.1.1 1.1. The Toy Language Trap

Visual block platforms (Scratch, Blockly) succeed at eliminating syntax errors, but they operate in an isolated sandbox. They lack first-class functions, lexical closures, structured error handling, dynamic arrays, hash maps, and real filesystem I/O. When students outgrow blocks, virtually none of their syntactic knowledge transfers to text.

2.1.2 1.2. The Industrial Cliff

When learners transition from blocks to textual programming, they are thrown into industrial languages designed for enterprise production rather than human learning: * **CPython**: While syntactically approachable, CPython is an impenetrable 500,000-line monolith of C code with complex reference-counting gymnastics, cyclic garbage collection heuristics, and GIL locks. Python hides memory layout, call stacks, and bytecode mechanics, leaving students with no mental model of how a computer actually works. * **Java & C++**: Learners are assaulted by punctuation noise, mandatory boilerplate (`public static void main`), confusing compiler diagnostic cascades, and complex build tools. * **Rust**: While safe and modern, Rust’s borrow checker imposes an insurmountable cognitive load on beginners who have not yet internalized basic control flow.

2.1.3 1.3. The Black Box Deception

High-level languages encourage students to view the computer as a magical black box. Values exist without memory addresses; functions execute without stack frames; collections grow without dynamic allocations; objects dispatch without method tables.

When these students later encounter computer architecture and systems programming courses, they suffer massive cognitive dissonance. They are forced to switch to C, where uninitialized pointers, segmentation faults, and undefined behavior consume their learning time.

2.2 Chapter 2: The Glass Box Paradigm

Unfish replaces the “Black Box” with the **Glass Box Paradigm**: > “*Complexity is hidden until the learner is ready—but it is never locked behind an opaque wall.*”

In Unfish, every running script can be inspected at any level of abstraction using standard command-line tools:

```
+-----+
| LEVEL 1: High-Level Unfish Source Code                               |
|   say 40 + 2                                                         |
+-----+
| LEVEL 2: Lexical Token Stream (`unfish tokens`)                     |
|   [TOK_IDENTIFIER "say"], [TOK_NUMBER 40], [TOK_PLUS "+"],         |
|   [TOK_NUMBER 2], [TOK_EOF]                                         |
+-----+
| LEVEL 3: Abstract Syntax Tree (`unfish ast`)                         |
|   (program (say (+ (literal 40) (literal 2))))                     |
+-----+
| LEVEL 4: Semantic Analysis & Scope Binding (`unfish check`)         |
```

```

|   Resolved: 'say' -> Global Builtin | Type: Number -> Number -> Number |
+-----+
| LEVEL 5: Stack Bytecode ISA (`unfish disasm`) |
| 0000 OP_CONSTANT      0 (40.0) |
| 0003 OP_CONSTANT      1 (2.0) |
| 0006 OP_ADD |
| 0007 OP_SAY |
| 0008 OP_RETURN |
+-----+
| LEVEL 6: Register VM 3-Address Code (`unfish disasm --reg`) |
| 0000 ROP_LOAD_K      R1, K0      ; R1 = 40.0 |
| 0001 ROP_LOAD_K      R2, K1      ; R2 = 2.0 |
| 0002 ROP_ADD          R0, R1, R2  ; R0 = R1 + R2 |
| 0003 ROP_SAY          R0 |
| 0004 ROP_RETURN |
+-----+
| LEVEL 7: Native C99 Transpilation (`unfish emit-c`) |
|   UfValue r1 = uf_val_number(40.0); |
|   UfValue r2 = uf_val_number(2.0); |
|   UfValue r0 = uf_val_number(r1.as.number + r2.as.number); |
|   uf_rt_say(rt, r0); |
+-----+
| LEVEL 8: Bare-Metal Machine Code (`unfish build --embedded --arm`) |
|   MOVS R1, #40 |
|   ADDS R0, R1, #2 |
|   BL   uart_print_number |
+-----+

```

The learner can start at Level 1 and gradually peel back each layer as their curiosity and curriculum advance. There are no sudden leaps, no jarring language switches, and no unlearning required.

3 PART II: LANGUAGE TOUR & CORE PROGRAMMING

3.1 Chapter 3: Lexical Structure, Indentation & Syntax

3.1.1 3.1. Clean Indentation Without Braces

Unfish enforces block structure using clean indentation (the off-side rule). A block begins with a colon `:` followed by an indented body:

```
let depth = 50

if depth > 30:
    say "Deep water dive"
    say "Check oxygen levels"
else:
    say "Shallow lagoon dive"
```

3.1.2 3.2. Comments & Docstrings

- Single-line comments start with `#`.
- Documentation docstrings start with `##` and attach directly to the following declaration, making them visible to `unfish doc` and the Language Server Protocol (LSP).

3.2 Chapter 4: Variables, Types & Control Flow

3.2.1 4.1. Variables with `let`

Variables are explicitly bound using `let`:

```
let name = "Lagoon"
let radius = 15.5
let active = true
```

3.2.2 4.2. Formatted String Interpolation (`f"..."`)

```
let fish_id = 7
let speed = 1.8
say f"Fish #{fish_id} moving at speed {speed} knots"
```

3.2.3 4.3. Loop Constructs

- **`repeat count::`** Dedicated educational loop executing a fixed number of times.
- **`while condition::`** Standard conditional while loop.
- **`for item in iterable::`** For-in loop over arrays, maps, strings, and ranges.

3.3 Chapter 5: Functions, Closures & `fn` Lambdas

Functions are first-class citizens. They can be created using `function` or the concise `fn` keyword:

```
# Named function
fn add(a, b):
    return a + b

# Concise lambda expression
```

```
let nums = [1, 2, 3, 4]
let doubled = map(nums, fn(x): x * 2)
say doubled # [2, 4, 6, 8]
```

3.3.1 Lexical Closures

Functions capture variables from enclosing scopes. Captured variables (upvalues) remain alive even after the outer function exits:

```
fn make_counter(start):
  let count = start
  return fn():
    count += 1
    return count

let c = make_counter(10)
say c() # 11
say c() # 12
```

3.4 Chapter 6: Structs, Methods & Ocean Simulation

Structs define custom data types with fields and associated methods. Position-based constructors are automatically synthesized:

```
## [Fish] School of Fish Simulation
## Simulating fish movement, vectors and distance in the ocean!

struct Fish:
  id
  x
  y
  speed

  fn swim(self, dx, dy):
    return Fish(self.id, self.x + dx * self.speed, self.y + dy * self.speed, self.speed)

  fn distance_from_reef(self):
    return sqrt(pow(self.x, 2) + pow(self.y, 2))

let school = [
  Fish(1, 10, 20, 1.5),
  Fish(2, 14, 22, 1.2),
  Fish(3, 8, 19, 1.8)
]

say "[Wave] Initial Fish Positions in the Lagoon:"
for f in school:
  say f"Fish #{f.id}: pos=({f.x}, {f.y}) - distance to reef={round(f.distance_from_reef())}"

say "\n[Swim] A current pushes the school by (dx=5, dy=3):"
let moved_school = map(school, fn(f): f.swim(5, 3))
for f in moved_school:
  say f"Fish #{f.id}: now at ({f.x}, {f.y}) - new distance={round(f.distance_from_reef())}"
```

3.5 Chapter 7: Structured Exception Handling

Unfish provides deterministic, structured exception handling with `try`, `catch`, and optional `finally`:

```
fn read_config(path):  
    try:  
        if not fs.exists(path):  
            raise f"Config file '{path}' not found!"  
        return fs.read_file(path)  
    catch err:  
        say f"Handled error: {err}"  
        return "{}"  
    finally:  
        say "Configuration load attempt complete."
```

3.5.1 The Invariants of Exception Unwinding

1. Stack frames are cleanly unwound back to the handler depth.
 2. The operand evaluation stack is restored.
 3. Code inside `finally` is guaranteed to execute, even on early returns, breaks, or unhandled exceptions.
-

4 PART III: SYSTEMS PROGRAMMING & COMPILER INTERNALS

4.1 Chapter 8: Contiguous Byte Buffers & Endian Codecs

Unlike Python which obscures memory, Unfish provides direct byte-level manipulation via contiguous `buffer` objects:

```
# Create a 32-byte binary packet
let packet = buffer_create(32)

# Write binary fields with explicit endianness
buffer_write_u32(packet, 0, 0x55464953, true) # Magic: 'UFIS' (Little Endian)
buffer_write_u16(packet, 4, 100, true)        # Sequence ID
buffer_write_f64(packet, 6, 98.6, true)       # Temperature Sensor Float

# Read binary fields
let magic = buffer_read_u32(packet, 0, true)
say f"Magic Header: {magic}"
```

4.2 Chapter 9: The Dual Virtual Machine Architecture

Unfish includes two virtual machines running in 100% differential parity lockstep:

57-Opcode Stack VM	256-Register Computed-Goto VM
- Classic 1-address stack model	- Modern 3-address register model
- Explicit push / pop operations	- 256 virtual registers per frame
- Ideal for introductory courses	- Computed-goto dispatch loop
- Clear operand stack traces	- 45%--60% faster execution

Both virtual machines share the same memory manager, object headers, garbage collector, and standard library.

4.3 Chapter 10: Ahead-of-Time C99 Transpilation & Bare-Metal Hardware

Unfish scripts can be compiled directly into standalone C99:

```
unfish emit-c script.unfish -o script.c
gcc -O3 script.c -lm -o script_native
```

This produces an independent binary that links with `src/codegen/unfish_runtime.h` and has **zero dependencies on the Unfish interpreter binary**.

4.3.1 Flashing Bare-Metal Microcontrollers

```
unfish build --embedded --arm sensor.unfish -o firmware.elf
```

Compiles with `-DUF_EMBEDDED`, eliminating host OS requirements and enabling execution on ARM Cortex-M microcontrollers.

5 APPENDIX: FORMAL GRAMMAR & VM OPCODES

5.0.1 Core EBNF Grammar Excerpt

```
Program      = { TopLevelDeclaration | Statement } EOF ;
Block        = ':' NEWLINE INDENT Statement { Statement } DEDENT ;
FunctionDecl = ( 'function' | 'fn' ) IDENTIFIER '(' [ ParamList ] ')' Block ;
StructDecl   = 'struct' IDENTIFIER ':' NEWLINE INDENT { StructMember } DEDENT ;
LambdaExpr   = ( 'function' | 'fn' ) '(' [ ParamList ] ')' ( Block | ':' Expression ) ;
PipeExpr     = LogicalOrExpr { '|' LogicalOrExpr } ;
```

5.0.2 Complete Test Parity Verification Ledger

All 95 conformance tests in `tests/conformance/` pass with 100% differential parity across all five execution engines: * 68 Feature Tests (`01_hello.unfish` through `68_fish_simulation.unfish`) * 27 Error Diagnostics Tests (`err_arity_mismatch` through `err_terminated_string`) * 0 Memory Leaks verified under `AddressSanitizer` and `LeakSanitizer`.